Fusion Is Composition: Kernel Fusion as the Functor Law

Hanzo AI — ML Systems

hanzo-kernel Paper 07.1 — concept core of the *One Source, Every Backend* series

Abstract

Kernel fusion is usually presented as a compiler optimization: a pass that inspects a graph and, when a cost model approves, merges adjacent operations to avoid writing an intermediate array to memory. This paper takes the opposite stance — fusion is not a pass, it is an *algebraic identity*. An elementwise GPU kernel is the lift of a scalar function to every array index, and such lifts obey the functor law $\mathrm{map}(g) \circ \mathrm{map}(f) = \mathrm{map}(g \circ f)$. Read right-to-left, that law *is* fusion: one kernel computing the composed function, no intermediate. Reduce and Scan are the points where the law stops applying — they are algebraic *fences*, decidable from the operation's class, not from a pattern matcher. We show how this single idea becomes a working, one-launch fusion engine in `hanzo-kernel`: a tiny op algebra whose fusibility rule is a total function of op class; one generic kernel that interprets any composed chain as straight-line arithmetic; and the same algebra lifted from a line to an arbitrary DAG. Every claim is gated bit-exactly on a CPU reference. The concrete payoff is measured on a real model pattern — the per-layer Gated-DeltaNet chain fuses from 9 kernel launches to 5, byte-for-byte identical to the unfused path.

1 The pain: an intermediate nobody wanted

Consider the tail of a Transformer feed-forward block, after its two matmuls have produced two arrays `gate` and `up`, each of length n. The SwiGLU activation is

$$y \;=\; \mathrm{silu}(\texttt{gate}) \cdot \texttt{up}, \qquad \mathrm{silu}(x) = x \cdot \sigma(x).$$

A *kernel* is a function the GPU runs once per array element in parallel; a kernel *launch* is one dispatch of that function over the whole array. The obvious way to compute y is two kernels: one computes $t = \mathrm{silu}(\texttt{gate})$, writing n floats to device memory (DRAM); a second reads t and `up` back from DRAM and writes y. That middle array t is *materialized* — written to DRAM and immediately read back — for no reason but that we wrote two kernels. On a decode step, where the whole workload is memory-bound (you are limited by how fast you can move bytes, not by arithmetic), that round-trip is pure waste.

Now scale the pattern. Qwen3.5 and Qwen3.6 are hybrid models whose bulk layers are Gated-DeltaNet (GDN) linear attention, and each GDN layer emits a short elementwise chain — $\mathrm{rms\_norm}(x) \cdot w \cdot \mathrm{silu}(g)$ and its siblings — as a *storm* of tiny kernel launches: roughly **9 launches per layer** on the ops-composed CUDA path. Each launch reads its inputs from DRAM, does a trivial amount of arithmetic, and writes its output back to DRAM for the next launch to read. Multiply by dozens of layers and every decoded token pays for a parade of intermediates. This is not a corner case; it is a measurable part of why an ops-composed engine trails a hand-fused one.

The question this paper answers is: *when may two adjacent kernels be merged into one, and how do you do it once, correctly, for every backend?* The usual answer is "run a fusion pass with a pattern matcher and a cost model." Ours is simpler and total.

2 The idea: an elementwise kernel is a lifted function

Strip a pointwise (elementwise) kernel to its essence and it is a scalar function $f : F \to F$ applied independently at every index i:

$$\mathrm{map}(f) \ : \ \mathrm{Array} \to \mathrm{Array}, \qquad \mathrm{map}(f)(A)[i] = f(A[i]).$$

The word *map* is exact: it is the same map a Rust programmer writes as `v.iter().map(f)`. Two such kernels in sequence apply f, store the result, then apply g. But applying g after f at every index is, by definition, applying $g \circ f$ at every index. That is the **functor law**:

$$\boxed{\mathrm{map}(g) \circ \mathrm{map}(f) \ = \ \mathrm{map}(g \circ f)}$$

Read left-to-right, the equation is two kernels with a materialized array between them. Read right-to-left, it is *one* kernel computing the composed function $g \circ f$ with *no* intermediate. **Kernel fusion is nothing but that law, read right-to-left.**

If this feels familiar it is the transducer insight in disguise: `map(f)` and `map(g)` are transformations that do not mention the collection they run over, so you may compose the *transformations* and apply the composite to the collection *once*, never allocating the intermediate collection. Here the collection is a GPU array and "once" is a single kernel launch. Categorically, each Map is a morphism Array $\to$ Array and *composition of morphisms is fusion.* There is no separate "fusion pass" to invent; there is only the associativity of $\circ$.

3 The class decides fusibility

Not every op is a Map, and the ones that are not are exactly the ones that cannot be fused this way. The whole surface is two classes:

- **Map** (a pointwise morphism): output element i depends only on input(s) at index i (plus compile-time constants). It is shape-preserving and *index-local.* Maps compose freely: any run of adjacent Maps is one Map, by the functor law. This is the fusible family — `Add Sub Mul Div Neg Recip Abs Exp Rsqrt Sigmoid Tanh Silu Gelu`. (`Silu` and `Gelu` are not special cases: they are *defined* as compositions of the primitives, so they are already one composed morphism.)
- **Reduce / Scan** (a contraction): output element i depends on a *whole row or axis* — `sum`, `max`, `mean`, and the operations built on them (RMSNorm, softmax, matmul). This is *not* index-local. It cannot be absorbed into an adjacent Map by the same lift, because it needs cross-lane communication: threads must exchange partial results (through shared memory or a subgroup reduction) to compute one output. A Reduce is therefore a **fence**.

The decision "can these two adjacent ops fuse?" is then a total function of their class — no pattern matcher, no cost model:

$$\mathrm{fusible}(a, b) \ := \ \mathrm{is\_map}(a) \wedge \mathrm{is\_map}(b).$$

At a fence there is still fusion, just attached at the fence's *ports*: a Map run *feeding* the reduction fuses into its input read (a *prologue*), and a Map run *consuming* the reduction's result fuses into its output write (an *epilogue*). The reduction kernel itself is the seam where the pattern changes. RMSNorm is the canonical shape: `square`(Map, prologue) → `sum`(fence) → `scale-by-rsqrt`(Map, epilogue) — three regions, the two Map runs each collapsed to one.

4 The mechanism: reify, fold, interpret

Three small steps turn the algebra into a running engine. All of it lives in one crate module and is proven on a CPU reference (§6).

Reify. A pointwise expression is recorded as a tree of scalar morphisms rather than computed (Listing 1). `Cur` is the value flowing down the pipeline at the current element; `In(id)` is a side-input array read at the current index; `Const` is a compile-time scalar. Operator overloading makes building one read like the math it denotes. Building an `Expr` computes nothing — it records the trace.

Listing 1: The op algebra. The whole fusible surface is a tree of unary and binary pointwise morphisms over the current value, side inputs, and constants.

```
enum Expr { Cur, In(usize), Const(f32), Un(UnOp, Box<Expr>), Bin(BinOp, Box<Expr>, Box<Expr>) }

// Morphism composition: substitute `inner` for every `Cur` leaf. This is the ONLY operation
// the fusion pass needs -- map(g) . map(f) becomes map(g[Cur := f]).
fn compose_cur(&self, inner: &Expr) -> Expr {
    match self {
        Expr::Cur => inner.clone(),
        Expr::Un(op, a) => Expr::Un(*op, Box::new(a.compose_cur(inner))),
        Expr::Bin(op,a,b)=> Expr::Bin(*op, Box::new(a.compose_cur(inner)),
                                       Box::new(b.compose_cur(inner))),
        leaf => leaf.clone(),
    }
}
```

Fold. The fusion pass is one fold over the chain of steps: accumulate adjacent Maps by composing their expressions (`compose_cur` *is* $g[\texttt{Cur} := f]$), and cut a new region at every Reduce. That is the entire pass (Listing 2). Its output is a list of regions; the number of regions is the number of kernels, and it is strictly fewer than the number of steps whenever any two Maps were adjacent — that gap *is* the fusion.

Listing 2: The fusion pass: fold adjacent Maps by composing their expressions; cut at every Reduce. No matcher, no cost model — just the class.

```
fn fuse(&self) -> Vec<Region> {
    let (mut regions, mut acc) = (Vec::new(), None);
    for step in &self.steps {
        match step {
            Step::Map(e) => acc = Some(match acc.take() { // g after f ==> g[Cur := f]
                None => e.clone(), Some(prev) => e.compose_cur(&prev),
            }),
            Step::Reduce(r) => { if let Some(e) = acc.take() { regions.push(Region::Map(e)); }
                                 regions.push(Region::Reduce(*r)); }
        }
```

```
    }
    if let Some(e) = acc.take() { regions.push(Region::Map(e)); }
    regions
}
```

Interpret. A fused Map region is lowered to a small linear "op program" carried as a compile-time value, and *one* generic kernel walks that program in a compile-time-*unrolled* loop (Listing 3). Because the loop and the branch are resolved at compile time (the DSL's lowering engine monomorphizes on the program value), each instruction becomes straight-line arithmetic in one kernel body: the input is read once, threaded through every op, and written once. There are no intermediate arrays and exactly one launch, whatever the chain length. This is not a hand-written fixed kernel — any linear elementwise chain runs through this one `fused_interp`.

Listing 3: The one fused kernel. `prog` rides as a compile-time value, so `#[unroll]` plus a comptime `match` lower the whole chain to straight-line code: one launch, zero intermediates.

```
#[kernel(targets(cuda, metal, vulkan, webgpu, cpu), unchecked)]
pub fn fused_interp<F: Float>(x: &Array<F>, sides: &Sequence<Array<F>>,
                              out: &mut Array<F>, #[comptime] prog: Program) {
    let i = ABSOLUTE_POS;
    if i < out.len() {
        let mut acc = x[i];
        #[unroll]
        for step in 0..prog.instrs.len() {
            match comptime!(prog.instrs[step]) {
                Instr::Un(op) => acc = apply_un_dev::<F>(op, acc),
                Instr::BinIn(op, slot) => acc = apply_bin_dev::<F>(op, acc, sides.index(slot)[i]),
                Instr::BinConst(op, b) => { let c = comptime!(f32::from_bits(b));
                                            acc = apply_bin_dev::<F>(op, acc, F::new(c)) }
            }
        }
        out[i] = acc;
    }
}
```

The ergonomic surface is a builder that reads like the composition it denotes: `Fuse::new(a).mul(w).add(b).silu().run(c)` appends one instruction per call, compiles them to one program, and dispatches exactly one `fused_interp`. The constant leaf is stored as `f32::to_bits` so the program is `Eq + Hash` — the lowering engine keys one compiled kernel per distinct fused chain.

5 From a line to a DAG

A real forward pass is not a straight line. A value *fans out* (is consumed by several later ops) and a binary op *fans in* (reads two independently computed values). The expression $\mathrm{silu}(a) \cdot b + a$ reuses a twice; a linear chain cannot express it in one region, but a DAG can, and it still fuses to *one* kernel. The same algebra lifts from a path to a graph with no new ideas:

1. **Trace.** A `NodeId`-taping newtype records an arbitrary pointwise DAG as a `Vec<Node>` (operator overloads push nodes instead of computing). The node algebra is the *same* `UnOp`/`BinOp`/`Red` as the line case — one IR, reused.
2. **Partition.** Flood-fill maximal Map-only connected regions; cut every edge into a fence (a Reduce, or any non-elementwise op — matmul, gather, conv). This is the identical `Class` predicate from §3, applied to graph edges instead of list-adjacency.

3. **Lower.** Each Map region compiles to a small DAG program over a per-element *slot* array, and one generic `dag_interp` kernel interprets it. Fan-out is a slot read by several later instructions; fan-in is a binary reading two slots. This generalizes `fused_interp`'s single accumulator (a line) to n accumulators (a graph): one launch, zero intermediates, any DAG shape.

6 Evaluation: the bit-exact gate

Fusion must not change the numbers. Every fused chain is checked three ways on the CPU runtime, which executes a Map in program order and is therefore the definitional reference (the companion oracle paper, [5], makes this law and its cross-backend enforcement precise):

1. the fused *one* kernel,
2. the naïve N-kernel pipeline that materializes $N-1$ device intermediates,
3. a plain-Rust reference.

The device morphism table (`apply_un_dev`) is a line-for-line mirror of the host table (`apply_un`) — which is *why* fused equals reference bit-exactly, not merely within tolerance. The committed tests exercise the two patterns the llama.cpp fusion audit named plus a stress case (Table 1). The SwiGLU tail and the 3-op chain are *byte-identical* fused-versus- naïve; a 7-op mixed chain runs in one launch and matches its host oracle. And on the real engine pattern that motivated §1, the DAG path fuses the per-GDN-layer chain $\mathrm{rms\_norm}(x) \cdot w \cdot \mathrm{silu}(g)$ from **9 launches to 5**, bit-exact ($\max = 2.98 \times 10^{-8}$) against the unfused per-op path — draining the launch storm directly.

Chain	**Launches (naïve → fused)**	**Result vs naïve / reference**
$\mathrm{silu}(a) \cdot b$ (SwiGLU tail)	$2 \to 1$	byte-identical / $\leq$ 4.8e−7
$\mathrm{silu}(a \cdot w + b)$	$3 \to 1$	byte-identical / $\leq$ 4.8e−7
7-op mixed chain (all families)	$7 \to 1$	matches host oracle
$\mathrm{rms\_norm}(x) \cdot w \cdot \mathrm{silu}(g)$ (GDN, DAG)	$9 \to 5$	bit-exact (2.98e−8)

Table 1: The fusion gate. Fused one-kernel = naïve N-kernel = plain-Rust reference on the CPU runtime. The launch-count drop is the fusion; the byte-identity is the guarantee that it changed nothing but the launch count.

7 Why this is the right base

Fusibility here is a property of the operation's *type*, computed once, not a fact a matcher rediscovers for each new op combination. Adding an op adds one algebra variant and its device mirror; it adds *zero* fusion rules. And the fusion composes *with* the peak kernels rather than competing with them: the hand-tuned reduction and matmul kernels stay, and the Map regions attach to them as read-prologue and write-epilogue at the fence. That is the whole point of making Reduce a first-class fence rather than something a fusion pass must be taught to stop at. The elementwise fabric of an entire model — the SwiGLU tails, the norm epilogues, the GDN gating — collapses to a minimal set of kernels by one identity, and every step is proven to change only the launch count, never the result.

References

[1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL.* Paper 07 (umbrella).

[2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law.* Paper 07.1 (this paper).

[3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning.* Paper 07.2.

[4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch.* Paper 07.3.

[5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law.* Paper 07.4.

[6] Hanzo AI. *When the Product Won't Collapse: An int8 Tensor-Core MMQ Negative Result.* Paper 07.5.

[7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product.* Paper 07.6.